

项目中上拉下拉刷新的实现

项目中上拉下拉刷新的实现方式

一、核心依赖库

项目使用 **MJRefresh 第三方库** 实现上拉加载更多与下拉刷新功能。MJRefresh 是 iOS 开发中主流的列表刷新框架，通过为 `UITableView` 添加头部（下拉刷新）、底部（上拉加载）组件，简化刷新逻辑，支持自定义样式与回调。

二、下拉刷新实现

1. 核心原理

为 `UITableView` 的 `mj_header` 属性赋值一个 MJRefresh 头部组件（如 `MJRefreshNormalHeader`），当用户下拉列表达到触发阈值时，组件自动进入“刷新中”状态，并执行预设的回调方法（如请求最新数据）；数据加载完成后，需手动调用 `endRefreshing` 方法结束刷新动画。

2. 关键代码示例（以 `ZYMainViewController` 为例）

（1）初始化下拉刷新组件（`viewDidLoad` 中）

```
- (void)viewDidLoad {
    [super viewDidLoad];
    // 其他初始化代码（如表格配置、数据数组初始化） ...

    // 1. 弱引用自身，避免 block 循环引用
    __weak typeof(self) weakSelf = self;

    // 2. 创建下拉刷新头部组件，并设置刷新回调
    self.tableView.mj_header = [MJRefreshNormalHeader headerWithRefreshingBlock:^(
        // 刷新回调：触发数据请求（获取最新数据）
```

```

[weakSelf listCityUrl];
}];

// 可选：配置刷新组件样式（如文字、颜色）
MJRefreshNormalHeader *header = (MJRefreshNormalHeader *)self.tableView.mj_header;
header.stateLabel.font = [UIFont systemFontOfSize:12]; // 状态文字字体
header.lastUpdatedTimeLabel.font = [UIFont systemFontOfSize:10]; // 最后刷新时间字体
[header setTitle:@"下拉可以刷新" forState:MJRefreshStateIdle]; // 闲置状态文字
[header setTitle:@"松开立即刷新" forState:MJRefreshStatePulling]; // 拉动状态文字
[header setTitle:@"正在刷新..." forState:MJRefreshStateRefreshing]; // 刷新中文字
}

```

(2) 数据请求与刷新结束（listCityUrl 方法中）

```

- (void)listCityUrl {
    // 1. 构建请求参数（根据业务需求配置）
    NSDictionary *parameters = @{@"type": @"1", @"page": @1};

    // 2. 发送网络请求（获取最新数据）
    [ZYNetworkRequest postBeanNoPrompt:parameters
                     url:kCityListURL
                success:^(id json) {
        // 3. 解析数据（JSON→模型→数据数组）
        ZYNetWorkModel *model = (ZYNetWorkModel *)json;
        if ([model.status isEqualToString:@"1"]) {
            NSArray *dataArray = (NSArray *)model.data;
            // 清空旧数据，添加新数据（下拉刷新通常是“覆盖更新”）
            [self.cityArray removeAllObjects];
            [self.cityArray addObjectsFromArray:dataArray];
        }

        // 4. 主线程更新UI+结束刷新
        dispatch_async(dispatch_get_main_queue(), ^{
            [self.tableView reloadData]; // 刷新表格，显示新数据
            [self.tableView.mj_header endRefreshing]; // 结束刷新动画（关键）
        });
    } failure:^(NSError *error) {
        // 5. 请求失败时，同样需要结束刷新
        dispatch_async(dispatch_get_main_queue(), ^{

```

```
[self.tableView.mj_header endRefreshing];
[MBProgressHUD showError:@"刷新失败，请重试"]; // 错误提示
});
} progress:^(float progress) {}];
}
```

(3) 页面显示时自动触发刷新（viewWillAppear 中）

项目中配置了“页面即将显示时自动刷新”，确保用户看到最新数据：

```
- (void)viewWillAppear:(BOOL)animated {
    [super viewWillAppear:animated];
    // 自动触发下拉刷新（无需用户手动下拉）
    [self.tableView.mj_header beginRefreshing];
}
```

三、上拉加载更多实现

1. 说明

从当前搜索结果来看，项目未完整展示上拉加载更多的代码，但基于 MJRefresh 的标准用法，上拉加载通过 UITableView 的 mj_footer 属性实现，逻辑与下拉刷新类似，核心是“滚动到底部触发加载→请求分页数据→追加到现有数组”。

2. 标准实现代码（参考）

(1) 初始化上拉加载组件（viewDidLoad 中）

```
- (void)viewDidLoad {
    [super viewDidLoad];
    // 其他初始化代码...

    __weak typeof(self) weakSelf = self;
    // 1. 创建上拉加载底部组件（自动加载：滚动到底部自动触发）
    self.tableView.mj_footer = [MJRefreshAutoNormalFooter footerWithRefreshingBlock:^(
```

```

// 加载更多回调：请求下一页数据
[weakSelf loadMoreCityData];
});

// 2. 配置加载组件样式
MJRefreshAutoNormalFooter *footer = (MJRefreshAutoNormalFooter *)self.tableView.mj_footer;
footer.stateLabel.font = [UIFont systemFontOfSize:12];
[footer setTitle:@"上拉加载更多" forState:MJRefreshStateIdle];
[footer setTitle:@"正在加载..." forState:MJRefreshStateRefreshing];
[footer setTitle:@"没有更多数据了" forState:MJRefreshStateNoMoreData]; // 无数据状态
}

```

(2) 加载更多数据（loadMoreCityData 方法中）

```

// 定义分页参数（在.h文件或.m私有属性中声明）
@property (nonatomic, assign) NSInteger currentPage; // 当前页码（从1开始）
@property (nonatomic, assign) NSInteger pageSize; // 每页条数（如20）
@property (nonatomic, assign) BOOL hasMoreData; // 是否还有更多数据
- (void)loadMoreCityData {
    // 1. 先判断是否还有更多数据（避免无效请求）
    if (!self.hasMoreData) {
        [self.tableView.mj_footer endRefreshingWithNoMoreData]; // 显示“无更多数据”
        return;
    }

    // 2. 构建分页请求参数（currentPage+1，请求下一页）
    NSDictionary *parameters = @{
        @"type": @"1",
        @"page": @(self.currentPage + 1),
        @"pageSize": @(self.pageSize)
    };

    // 3. 发送网络请求
    [ZYNetworkRequest postBeanNoPrompt:parameters
                        url:kCityListURL
                        success:^(id json) {
        ZYNetWorkModel *model = (ZYNetWorkModel *)json;
        if ([model.status isEqualToString:@"1"]) {

```

```

NSArray *newDataAry = (NSArray *)model.data;
// 4. 处理分页数据
if (newDataAry.count > 0) {
    // 有新数据：追加到现有数组，页码+1
    [self.cityArray addObjectsFromArray:newDataAry];
    self.currentPage++;
} else {
    // 无新数据：标记hasMoreData为NO，避免后续请求
    self.hasMoreData = NO;
}
}

// 5. 主线程更新UI+结束加载
dispatch_async(dispatch_get_main_queue(), ^{
    [self.tableView reloadData];
    if (self.hasMoreData) {
        [self.tableView.mj_footer endRefreshing]; // 正常结束加载
    } else {
        [self.tableView.mj_footer endRefreshingWithNoMoreData]; // 显示无更多数据
    }
});
} failure:^(NSError *error) {
    // 请求失败：结束加载，提示用户
    dispatch_async(dispatch_get_main_queue(), ^{
        [self.tableView.mj_footer endRefreshing];
        [MBProgressHUD showError:@"加载更多失败，请重试"];
    });
} progress:^(float progress) {});
}

```

四、刷新功能工作流程总结

无论是下拉刷新还是上拉加载，核心流程均为“初始化→触发→加载→更新→结束” 五步：

1. **初始化**：在 `viewDidLoad` 中为表格配置 `MJRefresh` 头部 / 底部组件，设置回调方法；

2. **触发**：

- 下拉刷新：用户下拉表格或页面显示时自动触发 (`beginRefreshing`) ；
- 上拉加载：用户滚动表格到底部时自动触发；

3. **加载数据**：在回调方法中发送网络请求，获取最新 / 下一页数据；
4. **更新 UI**：请求成功后，更新数据数组并刷新表格（`reloadData`）；
5. **结束刷新**：调用 `endRefreshing`（或 `endRefreshingWithNoMoreData`）停止动画，恢复表格正常状态。

该模式简洁高效，是 iOS 列表类页面的标准刷新实现方案，兼容性与可维护性强。

（注：文档部分内容可能由 AI 生成）